

Containerization with Docker

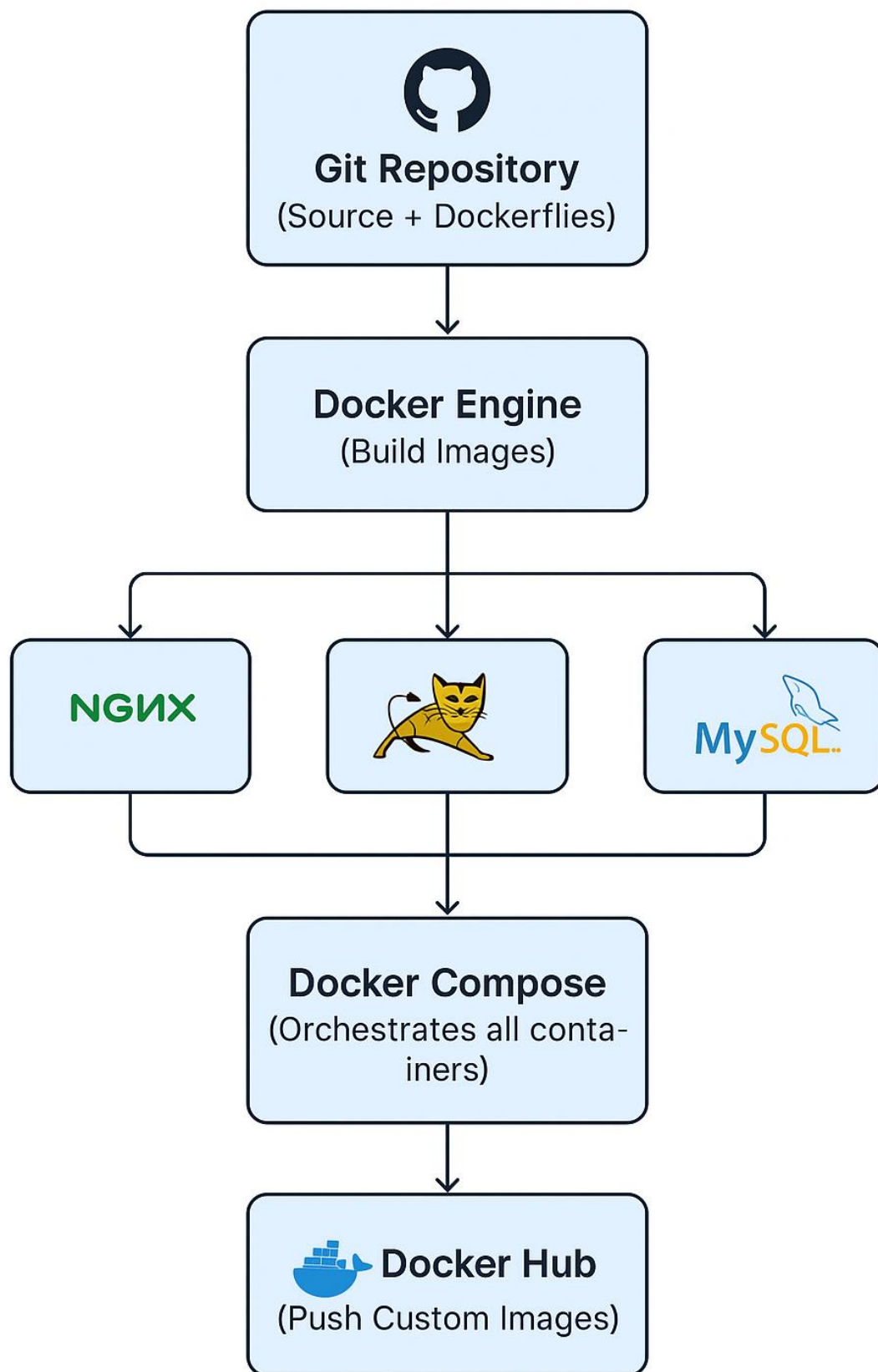
In this project, we are going to learn how to containerize an existing application using Docker.

Normally, applications run on virtual machines or physical servers, which leads to problems like:

- High cost and resource wastage (CPU/RAM not fully used).
- Environments (dev, QA, prod) not being in sync.
- Deployment issues and human errors.
- Difficulty running microservices on VMs.

Containers solve these problems because they use fewer resources, are portable, and run the same image in all environments. This makes deployments repeatable, faster, and cheaper.

ARCHITECTURE:



We will containerize the **VProfile application stack**, which has:

- Nginx
- Tomcat
- MySQL
- Memcached
- RabbitMQ

Tools

- Docker Engine – to build and run containers
- Dockerfile – to customize images
- Docker Compose – to run multi-container setup
- Docker Hub – to store and share images

Steps

1. Take the source code from Git.
2. Write Dockerfiles for services that need changes (Tomcat, Nginx, MySQL).
3. Build images using Docker.
4. Use Docker Compose to run all containers together.
5. Test locally.
6. Push images to Docker Hub.

Outcome

By the end, our application will run in containers instead of VMs. This will make it portable, cost-efficient, and consistent across all environments

1.Preparing for Containerization

Before writing Dockerfiles, the first thing we need is a **clear list of all services, tools, and their versions** used in the project. Without this, you can't decide which Docker images or tags to use.

In the **VProfile application**, the main services and tools are:

- **MySQL**
- **Tomcat**
- **Memcached**
- **RabbitMQ**
- **Nginx**
- (Plus Maven and JDK, which we need to build the artifact before deploying to Tomcat)

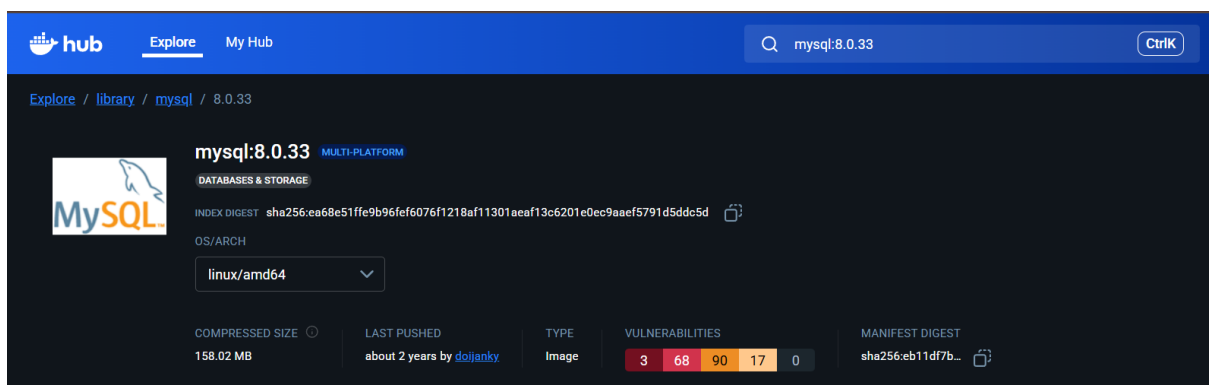
Why Versions Matter

When you containerize, you must use the same versions of services that were originally used in the project. For example:

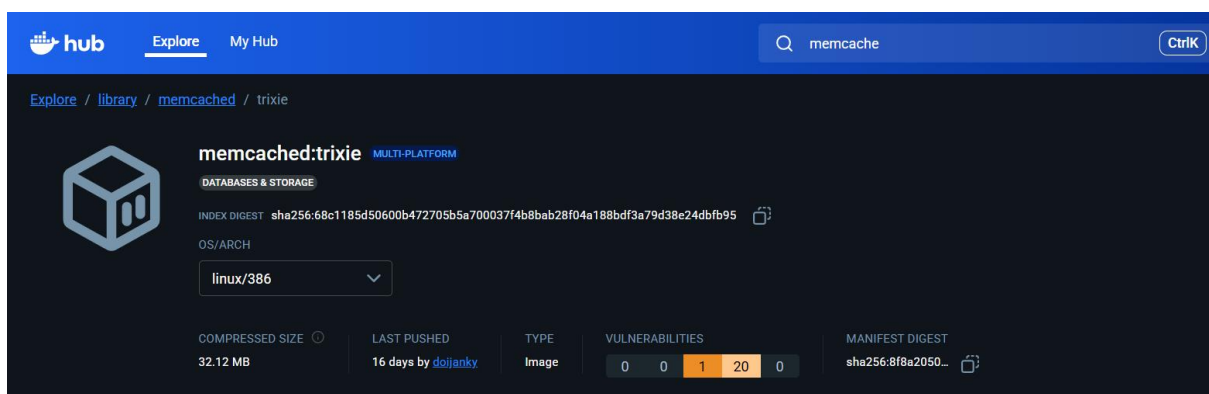
```
version: 0.2
ContainerizationServiceList (2).txt x
1 MySQL (Database SVC) => 8.0.33
2 Memcache (DB Caching SVC) => 1.6
3 RabbitMQ (Broker/Queue SVC) => 4.0
4 JDK => JDK 21
5 MAVEN => Maven 3.9.9
6 Tomcat (Application SVC) => 10, jdk21
7 Nginx (Web SVC) => 1.27
8
```

You can find these images and their tags on **Docker Hub**. Always match the version of your service to the correct tag. If the exact version isn't available, use the closest stable one and test it.

- `mysql:8.0.33`



- `memcache:latest`



- `rabbitmq:latest`

hub Explore My Hub

Search: rabbitmq CtrlK

Explore / library / rabbitmq / management

rabbitmq:management MULTI-PLATFORM

MESSAGE QUEUES

INDEX DIGEST sha256:85cb76f05fa216ce6e898522b4266351ef1bd9ac8a84373c7b0b8d125d6e87a9

OS/ARCH

linux/amd64

COMPRESSED SIZE 118.13 MB

LAST PUSHED 7 days by dojanky

TYPE Image

VULNERABILITIES 0 0 6 0

MANIFEST DIGEST sha256:02388e4b...

- maven:3.9.9-eclipse-temurin-21-jammy

hub Explore My Hub

Search: maven CtrlK

Explore / library / maven / 3.9.9-eclipse-temurin-21-jammy

maven:3.9.9-eclipse-temurin-21-jammy MULTI-PLATFORM

LANGUAGES & FRAMEWORKS

INDEX DIGEST sha256:7f8422bd81f19b9035007a75e2f82272b09af64db08848d7d4e9330246e8b6f

OS/ARCH

linux/amd64

COMPRESSED SIZE 226.4 MB

LAST PUSHED 4 months by dojanky

TYPE Image

VULNERABILITIES 0 1 26 19 0

MANIFEST DIGEST sha256:8e492952...

Layers (31) Vulnerabilities (47) Packages (277) [Give feedback](#)

- tomcat:10-jdk21

hub Explore My Hub

Search: tomcat CtrlK

Explore / library / tomcat / 10-jdk21

tomcat:10-jdk21 MULTI-PLATFORM

WEB SERVERS

INDEX DIGEST sha256:2056a11cee05402256e04648798e9f1313ee7290547ca984e6d64c491090aedd

OS/ARCH

linux/amd64

COMPRESSED SIZE 214.41 MB

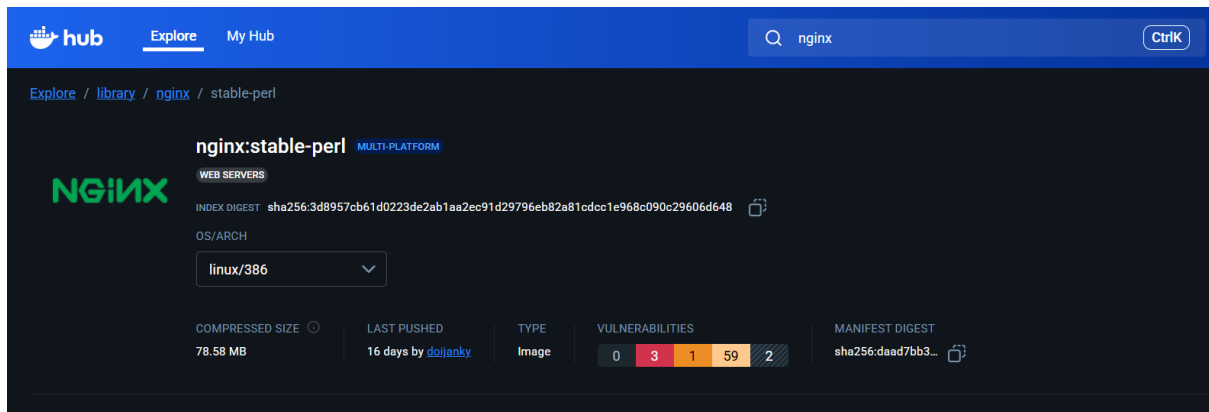
LAST PUSHED 8 days by dojanky

TYPE Image

VULNERABILITIES 0 0 12 10 0

MANIFEST DIGEST sha256:0a803800...

- nginx:latest



First Tasks

1. Review the **manual project setup** (VProfile Project Setup PDF or repo instructions). This helps you recall how each service was installed, configured, and connected.
2. Visit **Docker Hub** for each service and check:
 - Available image tags (versions).
 - How to configure (environment variables, volumes, passwords, etc.).
 - Doc link: https://github.com/sriramch163/vprofile-project/blob/local-VM/vagrant/Manual_provisioning_WinMacIntel/VprofileProjectSetupWindowsAndMacIntel.pdf

This preparation is important because when you write Dockerfiles later, you'll need to know what to customize:

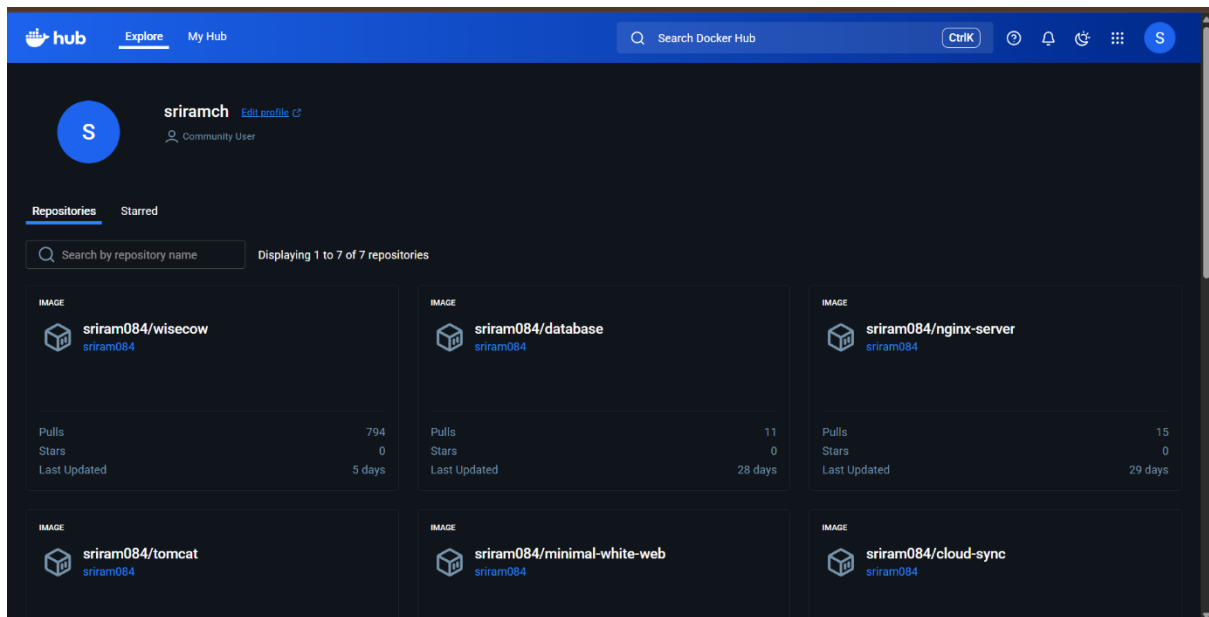
- **MySQL** → inject schema/data.
- **Tomcat** → add the built artifact.
- **Nginx** → add custom config.

Once this groundwork is done, we'll be ready to start writing Dockerfiles and building our custom images

2. Hosting Images on Docker Hub

Once we build our Docker images, we will host them on **Docker Hub**.

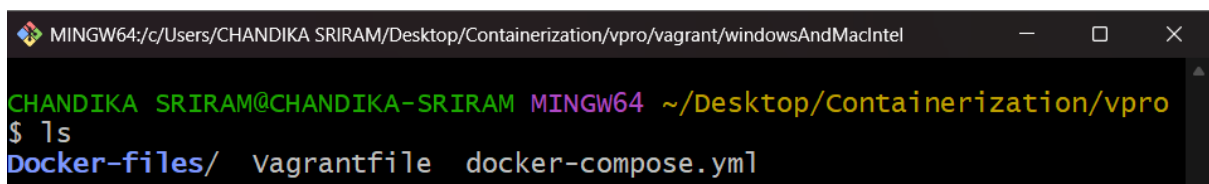
- **Create a Docker Hub Account**
- If you don't have one, sign up and log in at hub.docker.com



3.setting up the docker engine

Steps to achieve the goal

1. **Clone the source code**
 - Go to GitHub: <https://github.com/sriramch163/vprofile-project.git>.
 - Copy the HTTPS URL.
 - Open VSCode → Source Control → Clone Repository → paste URL.
 - Choose your destination folder
2. **Switch to the correct branch**
 - In VSCode, click the branch icon.
 - Select the **containers** branch.
 - You should now see the folders: Docker-files and vagrant.
3. **Navigate to Vagrant setup**
 - Open Git Bash (Windows) or Terminal (Mac).
 - Go to the cloned project folder → vagrant.
 - Inside, you will find folders for:
 - **Windows / Mac Intel**
 - Move into the appropriate folder based on your OS.



4. **Bring up the VM**
 - From the vagrant folder run:

```

CHANDIKA SRIRAM@CHANDIKA-SRIRAM MINGW64 ~/Desktop/Containerization/vpro
/vagrant/windowsAndMacIntel (containers)
$ vagrant up
Bringing machine 'default' up with 'virtualbox' provider...
==> default: This machine used to live in C:/Users/CHANDIKA SRIRAM/Desk
top/Containerization/vprofile-project/vagrant/windowsAndMacIntel but it
's now at C:/Users/CHANDIKA SRIRAM/Desktop/Containerization/vpro/vagran
t/windowsAndMacIntel.
==> default: Depending on your current provider you may need to change
the name of
==> default: the machine to run it as a different machine.
==> default: Checking if box 'ubuntu/jammy64' version '20241002.0.0' is
up to date...
==> default: Clearing any previously set forwarded ports...
==> default: Clearing any previously set network interfaces...
==> default: Preparing network interfaces based on configuration...
default: Adapter 1: nat
default: Adapter 2: hostonly
==> default: Forwarding ports...
default: 22 (guest) => 2222 (host) (adapter 1)
==> default: Running 'pre-boot' VM customizations...
==> default: Booting VM...
==> default: Waiting for machine to boot. This may take a few minutes..
.
default: SSH address: 127.0.0.1:2222

```

- Next ssh into the vm.

```

CHANDIKA SRIRAM@CHANDIKA-SRIRAM MINGW64 /d/xyz/vprofile-project/vagrant/windowsAndMacIntel (containers)
$ vagrant ssh
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-143-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Fri Sep 26 06:09:06 UTC 2025

System load:          1.46
Usage of /:           3.8% of 38.70GB
Memory usage:         9%
Swap usage:           0%
Processes:            116
Users logged in:      0
IPv4 address for enp0s3: 10.0.2.15
IPv6 address for enp0s3: fd17:625c:f037:2:17:d8ff:fe8d:76f6

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
New release '24.04.3 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

vagrant@ubuntu-jammy:~$

```

- And install the respective dependencies and repos of the docker.

```

• # Add Docker's official GPG key:
• sudo apt-get update
• sudo apt-get install ca-certificates curl
• sudo install -m 0755 -d /etc/apt/keyrings
• sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -
o /etc/apt/keyrings/docker.asc
• sudo chmod a+r /etc/apt/keyrings/docker.asc

```



```

• # Add the repository to Apt sources:
• echo \
• "deb [arch=$(dpkg --print-architecture) signed-
by=/etc/apt/keyrings/docker.asc]
https://download.docker.com/linux/ubuntu \
• $(. /etc/os-release && echo "${UBUNTU_CODENAME:-
$VERSION_CODENAME}") stable" | \
• sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
• sudo apt-get update
• sudo apt-get install docker-ce docker-ce-cli containerd.io
docker-buildx-plugin docker-compose-plugin

```

- And check the status of the docker whether it is running or not.

```

root@ubuntu-jammy:~# systemctl status docker
● docker.service - Docker Application Container Engine
   Loaded: loaded (/lib/systemd/system/docker.service; enabled; vendor preset: enabled)
   Active: active (running) since Fri 2025-09-26 06:13:09 UTC; 15s ago
 TriggeredBy: ● docker.socket
    Docs: https://docs.docker.com
   Main PID: 3363 (dockerd)
      Tasks: 9
     Memory: 21.9M
        CPU: 2.103s
   CGroup: /system.slice/docker.service
           └─3363 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

Sep 26 06:13:06 ubuntu-jammy dockerd[3363]: time="2025-09-26T06:13:06.888270072Z" level=info msg="detected 127.0.0.53 nameserver, assuming syste
Sep 26 06:13:07 ubuntu-jammy dockerd[3363]: time="2025-09-26T06:13:07.037741167Z" level=info msg="Creating a containerd client" address=/run/con
Sep 26 06:13:07 ubuntu-jammy dockerd[3363]: time="2025-09-26T06:13:07.167274362Z" level=info msg="Loading containers: start,"
Sep 26 06:13:08 ubuntu-jammy dockerd[3363]: time="2025-09-26T06:13:08.879488872Z" level=info msg="Loading containers: done."
Sep 26 06:13:08 ubuntu-jammy dockerd[3363]: time="2025-09-26T06:13:08.968684625Z" level=info msg="Docker daemon" commit=249d679 containerd-snaps
Sep 26 06:13:08 ubuntu-jammy dockerd[3363]: time="2025-09-26T06:13:08.969177314Z" level=info msg="Initializing buildkit"
Sep 26 06:13:09 ubuntu-jammy dockerd[3363]: time="2025-09-26T06:13:09.118572543Z" level=info msg="Completed buildkit initialization"
Sep 26 06:13:09 ubuntu-jammy dockerd[3363]: time="2025-09-26T06:13:09.131690042Z" level=info msg="Daemon has completed initialization"
Sep 26 06:13:09 ubuntu-jammy dockerd[3363]: time="2025-09-26T06:13:09.132759610Z" level=info msg="API listen on /run/docker.sock"
Sep 26 06:13:09 ubuntu-jammy systemd[1]: Started Docker Application Container Engine.
root@ubuntu-jammy:~#

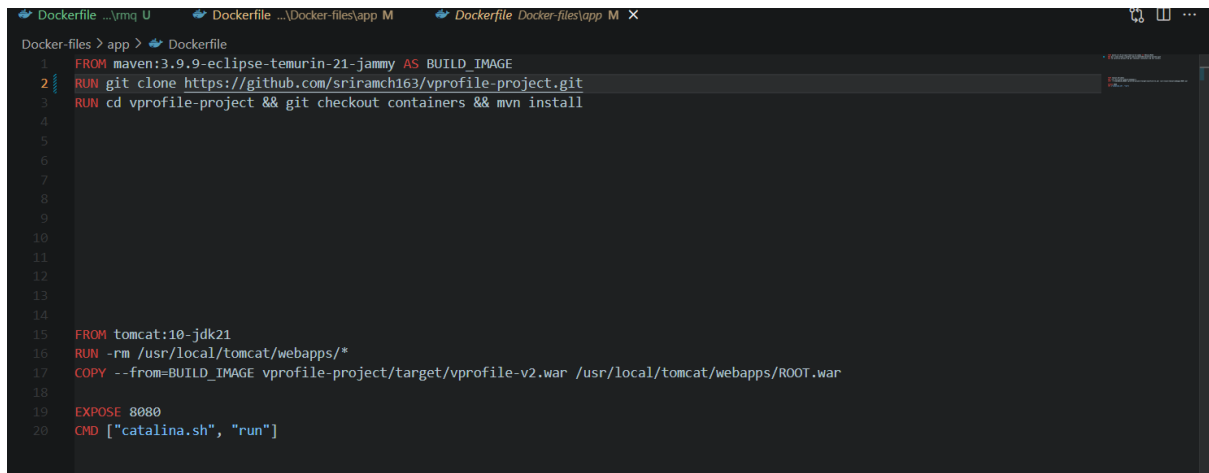
```

- And next create the repos in the you docker hub account.

Name	Last Pushed 	Contains	Visibility	Scout
sriram084/vprofileweb	30 minutes ago		Public	Inactive
sriram084/vprofileapp	30 minutes ago		Public	Inactive
sriram084/vprofilermq	30 minutes ago		Public	Inactive
sriram084/vprofilemc	31 minutes ago		Public	Inactive
sriram084/vprofiledb	33 minutes ago		Public	Inactive

4.Docker file for App

- And check the status of the docker whether it is running or not.
- Make sure when changing the names of the databases and the passwords of databases, make sure change in the application.properties files also. Then only it works.



```

1 FROM maven:3.9.9-eclipse-temurin-21-jammy AS BUILD_IMAGE
2 RUN git clone https://github.com/sriramch163/vprofile-project.git
3 RUN cd vprofile-project && git checkout containers && mvn install
4
5
6
7
8
9
10
11
12
13
14
15 FROM tomcat:10-jdk21
16 RUN -rm /usr/local/tomcat/webapps/*
17 COPY --from=BUILD_IMAGE vprofile-project/target/vprofile-v2.war /usr/local/tomcat/webapps/ROOT.war
18
19 EXPOSE 8080
20 CMD ["catalina.sh", "run"]

```

1. Build Stage (Maven & JDK)

```
FROM maven:3.9.9-eclipse-temurin-21-jammy AS BUILD_IMAGE
```

- Uses a Maven base image with Eclipse Temurin JDK 21 on Ubuntu Jammy.
- This stage is responsible for building the Java application.

```
RUN git clone https://github.com/sriramch163/vprofile-project.git
RUN cd vprofile-project && git checkout containers && mvn install
```

- Clones the vprofile-project repository from GitHub.
- Checks out the containers branch.
- Runs mvn install to build the project, producing a WAR file.

2. Runtime Stage (Tomcat)

```
FROM tomcat:10-jdk21
```

- Uses the official Tomcat 10 image with JDK 21.
- This is the lightweight runtime image where the WAR will be deployed.

```
RUN -rm /usr/local/tomcat/webapps/*
```

- Removes default web applications from Tomcat (/usr/local/tomcat/webapps).

```
COPY --from=BUILD_IMAGE vprofile-project/target/vprofile-v2.war
/usr/local/tomcat/webapps/ROOT.war
```

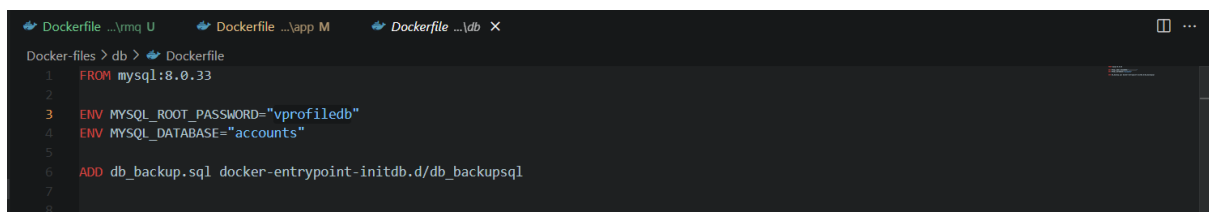
- Copies the generated vprofile-v2.war from the build stage into Tomcat as ROOT.war, making it the default application.

3. Container Configuration

```
EXPOSE 8080
CMD ["catalina.sh", "run"]
```

- Exposes port 8080 for accessing the Tomcat server.
- Starts Tomcat using the default `catalina.sh run` command.

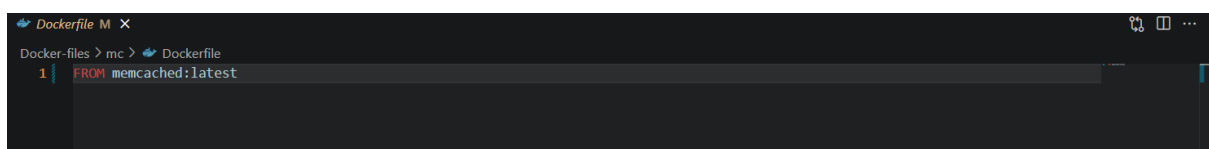
5. Docker file for Db



```
Dockerfile
1 FROM mysql:8.0.33
2
3 ENV MYSQL_ROOT_PASSWORD="vprofiledb"
4 ENV MYSQL_DATABASE="accounts"
5
6 ADD db_backup.sql docker-entrypoint-initdb.d/db_backups.sql
```

- **FROM mysql:8.0.33:** This line specifies the base image for the Docker container. It's using version 8.0.33 of the official MySQL image.
- **ENV MYSQL_ROOT_PASSWORD="vprofiledb":** This sets an environment variable named `MYSQL_ROOT_PASSWORD`, assigning the value "vprofiledb" as the password for the root user of the MySQL database.
- **ENV MYSQL_DATABASE="accounts":** This sets an environment variable named `MYSQL_DATABASE`, creating a new database named "accounts" when the container starts.
- **ADD db_backup.sql docker-entrypoint-initdb.d/db_backups.sql:** This command copies a local file named `db_backup.sql` into the container's `/docker-entrypoint-initdb.d/` directory. Files in this special directory are automatically executed by the MySQL image's entrypoint script when the container is first created. This means `db_backup.sql` will be run to initialize the database, likely to load a schema or some initial data

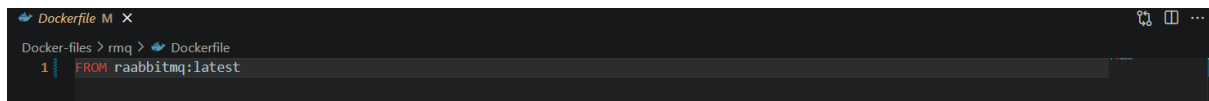
6. Docker file for MC



```
Dockerfile
1 FROM memcached:latest
```

- Which we takes the latest image

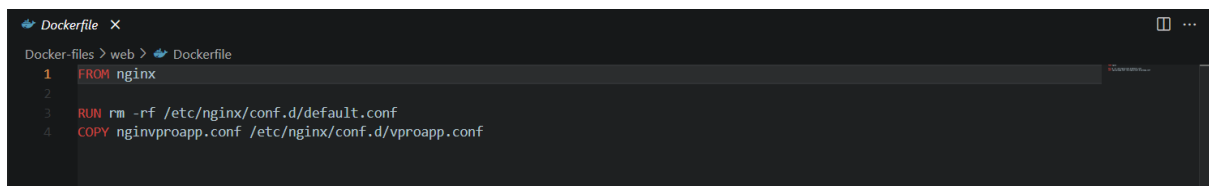
7. Docker file for RMQ

A screenshot of a code editor showing a Dockerfile. The file is named 'Dockerfile' and is located in a directory named 'rmq'. The content of the Dockerfile is:

```
1 FROM rabbitmq:latest
```

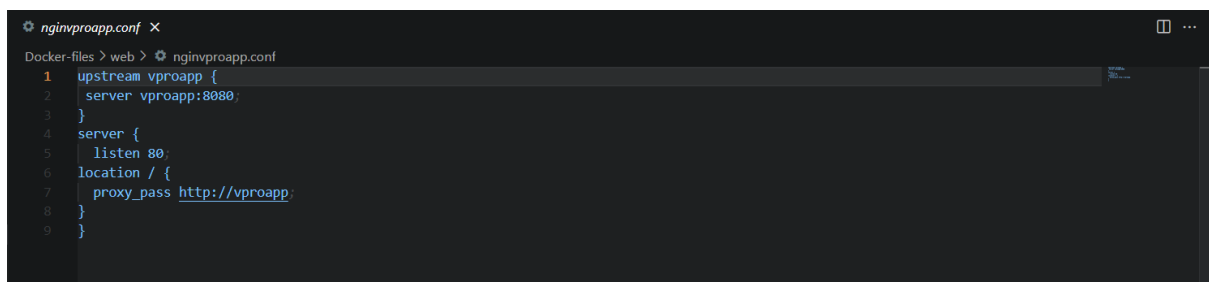
- Here also we taking the latest image.

8. Docker file for WEB

A screenshot of a code editor showing a Dockerfile. The file is named 'Dockerfile' and is located in a directory named 'web'. The content of the Dockerfile is:

```
1 FROM nginx
2
3 RUN rm -rf /etc/nginx/conf.d/default.conf
4 COPY nginxvproapp.conf /etc/nginx/conf.d/vproapp.conf
```

- **FROM nginx:** This is the base for the new container. It starts with the official `nginx` image, which includes everything needed to run the Nginx web server.
- **RUN `rm -rf /etc/nginx/conf.d/default.conf`:** This command is executed inside the container's build process. It removes the default Nginx configuration file. This is a common step when you want to replace the default settings with a custom one.
- **COPY `nginxvproapp.conf /etc/nginx/conf.d/vproapp.conf`:** This command copies a local file named `nginxvproapp.conf` from the same directory as the Dockerfile into the container. The destination path is where Nginx looks for configuration files, so this new file will be used to configure the web server. This file likely contains specific rules for routing traffic, serving static content, or acting as a reverse proxy for a web application.

A screenshot of a code editor showing the `nginxvproapp.conf` file. The file is located in a directory named 'web'. The content of the file is:

```
1 upstream vproapp {
2     server vproapp:8080;
3 }
4 server {
5     listen 80;
6     location / {
7         proxy_pass http://vproapp;
8     }
9 }
```

which is an Nginx configuration file.

This configuration sets up **Nginx to act as a reverse proxy**. Here's a breakdown of the code:

- **upstream vproapp { ... }:** This block defines a group of upstream servers (in this case, just one).
 - **server vproapp:8080;** This line specifies the address of the backend application server. Nginx will forward requests to this server. The name `vproapp` suggests it's a hostname for a service within a Docker network, and `8080` is the port it's listening on.

- **server { ... }:** This block defines a virtual server that Nginx will use to listen for incoming requests.
 - **listen 80;:** This tells Nginx to listen for incoming web traffic on port 80, the standard HTTP port.
 - **location / { ... }:** This block specifies what to do with requests for the root path (/).
 - **proxy_pass http://vproapp;:** This is the core of the reverse proxy configuration. It forwards all incoming requests that match the `location /` block to the upstream server group named `vproapp` (which is defined in the first block). Nginx will handle the request and pass it to the backend application running at <http://vproapp:8080>.

9. Docker Compose file



- **vprodb::** This defines a service named `vprodb`.
- **build::** This section instructs Docker Compose to build a Docker image from a Dockerfile rather than pulling a pre-existing image.
 - **context: ./Docker-files/db:** The build context is set to the `./Docker-files/db` directory. This is where Docker will look for the Dockerfile to build the image.
- **image: sriram84/vprofiledb:** After the image is built, it will be tagged as `sriram84/vprofiledb`. This is useful for pushing the image to a Docker registry.
- **container_name: vprodb:** The running container will be named `vprodb`. This makes it easier to reference and manage.
- **ports::** This section maps ports between the host machine and the container.
 - **"3306:3306":** This maps port 3306 on the host to port 3306 inside the container. This allows the MySQL database to be accessed from the host machine on the standard MySQL port.
- **volumes::** This section defines a volume to persist data.

- **- vproddbdata:/var/lib/mysql:** This creates a named volume called `vproddbdata` and mounts it to the `/var/lib/mysql` directory inside the container. This directory is where MySQL stores its data, ensuring that the database data is not lost when the container is stopped or removed.
- **environment::** This section sets environment variables for the container.
 - **- MYSQL_ROOT_PASSWORD=vprofiledb:** This sets the `MYSQL_ROOT_PASSWORD` environment variable, which is used by the MySQL Docker image to set the root user's password. The value here matches the one specified in the associated Dockerfile.

```
vprocache01:
  build:
    context: ./Docker-files/mc
  image: sriram084/vprofilemc
  container_name: vpromc
  ports:
    - "11211:11211"
```

- **vprocache01::** This defines a service named `vprocache01`.
- **build::** This section indicates that Docker Compose should build an image for this service.
 - **context: ./Docker-files/mc:** The build context is set to the `./Docker-files/mc` directory, where the `Dockerfile` for the caching service is located.
- **image: sriram84/vprofilemc:** The built image will be tagged as `sriram84/vprofilemc`, which is the name used for the image after it's built.
- **container_name: vpromc:** The running container will be named `vpromc`. This provides a predictable name for easier management and networking.
- **ports::** This section maps ports between the host and the container.
 - **"11211:11211":** This maps port `11211` on the host machine to port `11211` inside the container. Port `11211` is the standard port for Memcached, a popular in-memory key-value store used for caching. This mapping allows the caching service to be accessed from the host.

```
vpromq01:
  build:
    context: ./Docker-files/rmq
  image: sriram084/vprofilermq
  container_name: vpromq
  ports:
    - "5672:5672"
  environment:
    - RABBITMQ_DEFAULT_USER=guest
    - RABBITMQ_DEFAULT_PASS=guest
```

- **vpromq01::** This defines a service named `vpromq01`.
- **build::** This section specifies that an image for this service will be built.
 - **context: ./Docker-files/rmq:** The build process will use the `Dockerfile` located in the `./Docker-files/rmq` directory.
- **image: sriram84/vprofilermq:** The resulting image will be tagged as `sriram84/vprofilermq`.
- **container_name: vpromq:** The running container will be named `vpromq` for easy reference and management.
- **ports::** This section maps ports.
 - **"5672:5672":** This maps port `5672` on the host to port `5672` inside the container. Port `5672` is the standard port for RabbitMQ's AMQP (Advanced Message Queuing Protocol) communication.
- **environment::** This section sets environment variables.
 - **- RABBITMQ_DEFAULT_USER=guest:** This sets the default username for RabbitMQ to `guest`.
 - **- RABBITMQ_DEFAULT_PASS=guest:** This sets the default password for the `guest` user to `guest`. These are default credentials for development environments.

```

vproapp:
  build:
    context: ./Docker-files/app
  image: sriram084/vprofileapp
  container_name: vproapp
  ports:
    - "8080:8080"
  volumes:
    - vproappdata:/usr/local/tomcat/webapps

```

- **vproapp::** This defines the service for the main application.
- **build::** This section instructs Docker Compose to build a custom image for this service.
 - **context: ./Docker-files/app:** The build process will use the `Dockerfile` located in the `./Docker-files/app` directory.
- **image: sriram84/vprofileapp:** The resulting image will be tagged as `sriram84/vprofileapp`.

- **container_name: vproapp:** The container will be given the specific name `vproapp`. This makes it easy to reference in other configurations (like the Nginx reverse proxy) and for manual management.
- **ports::** This maps a port from the container to the host machine.
 - **"8080:8080":** This maps port `8080` on the host to port `8080` inside the container. This is the standard port for Tomcat and web applications, allowing access to the application from outside the container.
- **volumes::** This section defines a volume for data persistence.
 - **- vproappdata:/usr/local/tomcat/webapps:** This creates a named volume `vproappdata` and mounts it to the `/usr/local/tomcat/webapps` directory inside the container. This directory is where Tomcat stores web applications, so this volume ensures that the application files and any related data are persistent even if the container is removed or replaced.

```

vproweb:
  build:
    context: ./Docker-files/web
    image: sriram084/vprofileweb
    container_name: vproweb
    ports:
      - "80:80"

volumes:
  vprodbdata: {}
  vproappdata: {}

```

- **vproweb::** This defines the service for the web server.
- **build::** This section specifies that an image for this service should be built from a Dockerfile.
 - **context: ./Docker-files/web:** The build process will use the Dockerfile located in the `./Docker-files/web` directory.
- **image: sriram84/vprofileweb:** The built image will be tagged as `sriram84/vprofileweb`.
- **container_name: vproweb:** The running container will be named `vproweb`.
- **ports::** This maps a port from the container to the host.
 - **"80:80":** This maps port `80` on the host machine to port `80` inside the container, making the web server accessible on the standard HTTP port.

Volume Definitions

- **volumes::** This section at the bottom of the file defines named volumes that can be used by services.
 - **vproddbdata: {}:** This defines a named volume called `vproddbdata`. The empty curly braces `{}` are a shorthand for an empty configuration, which means Docker Compose will handle the creation and management of the volume. This volume is used by the `vproddb` (database) service to persist its data.
 - **vproappdata: {}:** This defines a named volume called `vproappdata`. This volume is used by the `vproapp` (application) service to persist its web application files.

10. Docker Compose build

```
root@ubuntu-jammy:/vagrant# docker compose build
[+] Building 431.6s (38/40)
[+] Building 687.3s (38/40)
[+] Building 687.8s (38/40)
[+] Building 688.1s (38/40)
[+] Building 699.3s (42/42) FINISHED
=> [internal] load local bake definitions 0.0s
=> reading from stdin 2.22kB 0.0s
=> [vprocache01 internal] load build definition from Dockerfile 0.1s
=> => transferring dockerfile: 58B 0.1s
=> [vproddb internal] load build definition from Dockerfile 0.3s
=> => transferring dockerfile: 192B 0.2s
=> [vproapp internal] load build definition from Dockerfile 0.3s
=> => transferring dockerfile: 456B 0.2s
=> WARN: StageNameCasing: Stage name 'BUILD_IMAGE' should be lowercase (line 1) 0.3s
=> [vpromq01 internal] load build definition from Dockerfile 0.3s
=> => transferring dockerfile: 57B 0.3s
=> [vproweb internal] load build definition from Dockerfile 0.2s
=> => transferring dockerfile: 146B 0.2s
=> [vprocache01 internal] load metadata for docker.io/library/memcached:latest 2.3s
=> [vproweb internal] load metadata for docker.io/library/nginx:latest 2.3s
=> [vproapp internal] load metadata for docker.io/library/tomcat:10-jdk21 2.4s
=> [vproapp internal] load metadata for docker.io/library/maven:3.9.9-ecclipse-temurin-21-jammy 2.2s
=> [vproddb internal] load metadata for docker.io/library/mysql:8.0.33 2.2s
=> [vpromq01 internal] load metadata for docker.io/library/rabbitmq:latest 3.6s
=> [vprocache01 internal] load .dockerignore 0.2s
=> => transferring context: 2B 0.0s
=> [vproweb internal] load .dockerignore 0.2s
=> => transferring context: 2B 0.0s
=> [vproddb internal] load .dockerignore 0.2s
=> => transferring context: 2B 0.0s
=> [vproapp internal] load .dockerignore 0.2s
=> => transferring context: 2B 0.0s
=> [vprocache01 1/1] FROM docker.io/library/memcached:latest@sha256:68c1185d50600b472705b5a700037f4b8bab28f04a188bdf3a79d38e24dbfb95 26.3s
=> => resolve docker.io/library/memcached:latest@sha256:68c1185d50600b472705b5a700037f4b8bab28f04a188bdf3a79d38e24dbfb95 0.4s
=> => sha256:68c1185d50600b472705b5a700037f4b8bab28f04a188bdf3a79d38e24dbfb95 10.25kB / 10.25kB 0.0s
=> => sha256:857b814a2d185e48f488a0b57abe4af74c6b4b03f4c4ce32ab0d241ef702cc5d 2.11kB / 2.11kB 0.0s
=> => sha256:2cf3d5da134b2e02a0ebf4f4c0231e3644ab09f6961f87a90ba36fe6579ecfd6 4.55kB / 4.55kB 0.0s
=> => sha256:062b8c9a764a5b4060630b2c562795d278f9d556847f9b455468db4da51ea18a 1.10kB / 1.10kB 1.1s
=> => sha256:ce1261c6d567efa8e3b457673eeeb474a0a8066df6bb95ca9a6a94a31e219dd3 29.77MB / 29.77MB 10.4s
=> => sha256:185c70188d2a517b4cc82dcb27ac01c70e1ae81db07bela0f8876cba269daaa5 136.62kB / 136.62kB 1.7s
=> => sha256:f75ecf593ee29e25f3af5e7ceaad0c240832da7cb957a24a722fb66f7577dc87 2.29MB / 2.29MB 4.6s
=> => sha256:1131bb85eb4d149381872cf23566c235e019b5943b9e303c8ac0501045999aed 282B / 282B 2.3s
```

Based on the image you provided, the screenshot shows the output of a `docker compose build` command running in a Linux terminal. Here's a breakdown of what's happening:

Docker Build Process

The output log is detailing the steps taken to build a set of Docker images defined in a `docker-compose.yml` file.

- **[+] Building 431.6s (38/40):** This line, along with the subsequent similar lines, indicates the overall progress of the build. It shows the elapsed time and the number of build steps completed out of the total.
- **[+] Building 699.3s (42/42) FINISHED:** This marks the successful completion of the entire build process.

```

-> [vproapp] resolving provenance for metadata file 0.0s
[+] Building 5/5 0.1s
✓ sriram084/vprofilemq Built 0.0s
✓ sriram084/vprofileweb Built 0.0s
✓ sriram084/vprofileapp Built 0.0s
✓ sriram084/vprofilemc Built 0.0s
✓ sriram084/vprofiledb Built 0.0s

```

- This output signifies that the entire docker compose build operation has finished without any errors, and all the specified Docker images (vprofilemq, vprofileweb, vprofileapp, vprofilemc, and vprofiledb) are now ready to be used or pushed to a container registry.
- After the successful build, check the images by using the cmd.

```

root@ubuntu-jammy: /vagrant
root@ubuntu-jammy:/vagrant# docker images
REPOSITORY          TAG          IMAGE ID          CREATED          SIZE
sriram084/vprofileapp latest       c2c7e852d119     7 minutes ago   552MB
sriram084/vprofiledb latest       b1bb72d77d38     15 minutes ago  565MB
sriram084/vprofileweb latest       382bb718a4c9     17 minutes ago  192MB
sriram084/vprofilemq latest       658a9c7b6d12     2 weeks ago    234MB
sriram084/vprofilemc latest       4bc95b128d3b     6 weeks ago    85MB

```

- Next docker compose up. There it created the container , in that I shoes the container are to be created.

```

root@ubuntu-jammy:/vagrant# docker compose up -d
[+] Running 8/8
✓ Network vagrant_default Created 0.5s
✓ Volume vagrant_vproappdata Created 0.0s
✓ Volume vagrant_vprodbdata Created 0.0s
✓ Container vpromc Started 6.7s
✓ Container vprofileweb Started 6.1s
✓ Container vprofilemq Started 6.3s
✓ Container vproapp Started 7.4s
✓ Container vprodb Started 7.1s

```

- To check the containers are running use the command.. docker ps

```

root@ubuntu-jammy:/vagrant# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
db8f93a76eb9   sriram084/vprofiledb "docker-entrypoint.s..." 20 seconds ago Up 17 seconds 0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp, 3306/tcp
db091de43084   sriram084/vprofileapp "catalina.sh run"        20 seconds ago Up 17 seconds 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
b3505bb696c2   sriram084/vprofileweb "/docker-entrypoint.s..." 20 seconds ago Up 17 seconds 0.0.0.0:80->80/tcp, [::]:80->80/tcp
f34f25393ed0   sriram084/vprofilemc "docker-entrypoint.s..." 20 seconds ago Up 17 seconds 0.0.0.0:11211->11211/tcp, [::]:11211->11211/tcp
ch41423ad3d6   sriram084/vprofilemq "docker-entrypoint.s..." 20 seconds ago Up 17 seconds 4369/tcp, 5671/tcp, 15691-15692/tcp, 25672/tcp, 0.0.0.0:5672->5672/tcp, [::]:5672->5672/tcp

```

- There we can see that's the tomcat app is connected via nginx such as web, which is port 80.
- 0.0.0.0:80→80/tcp. This indicates when can access the application using the ip address.
- Now we can achieve this by using the cmd "ip addr show"

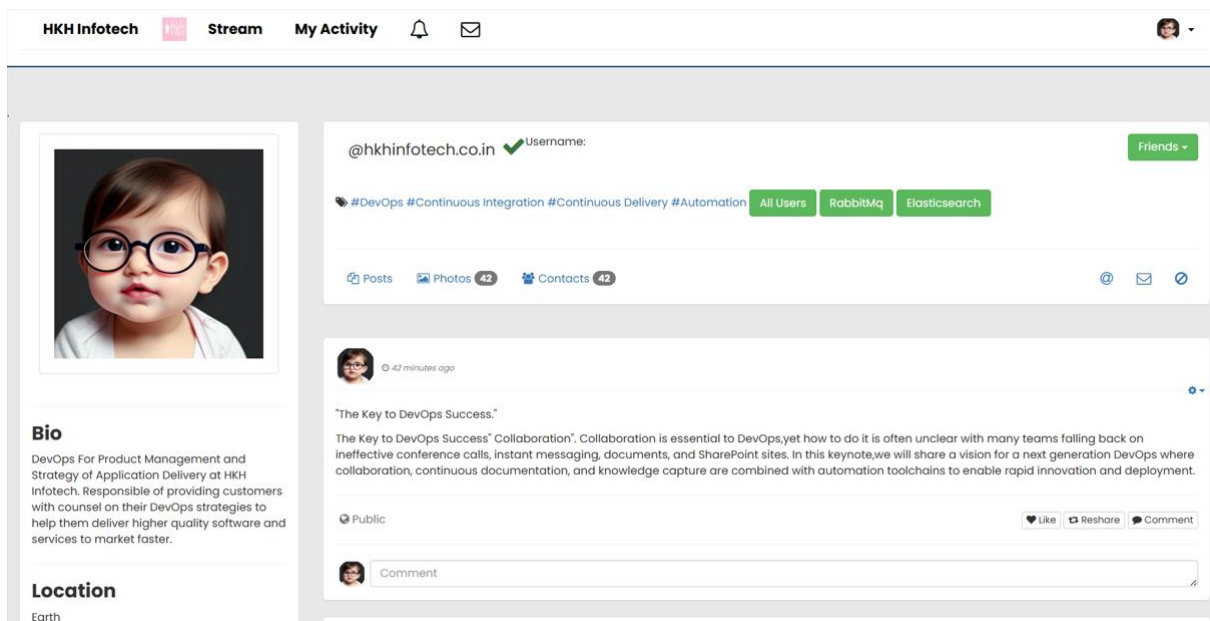
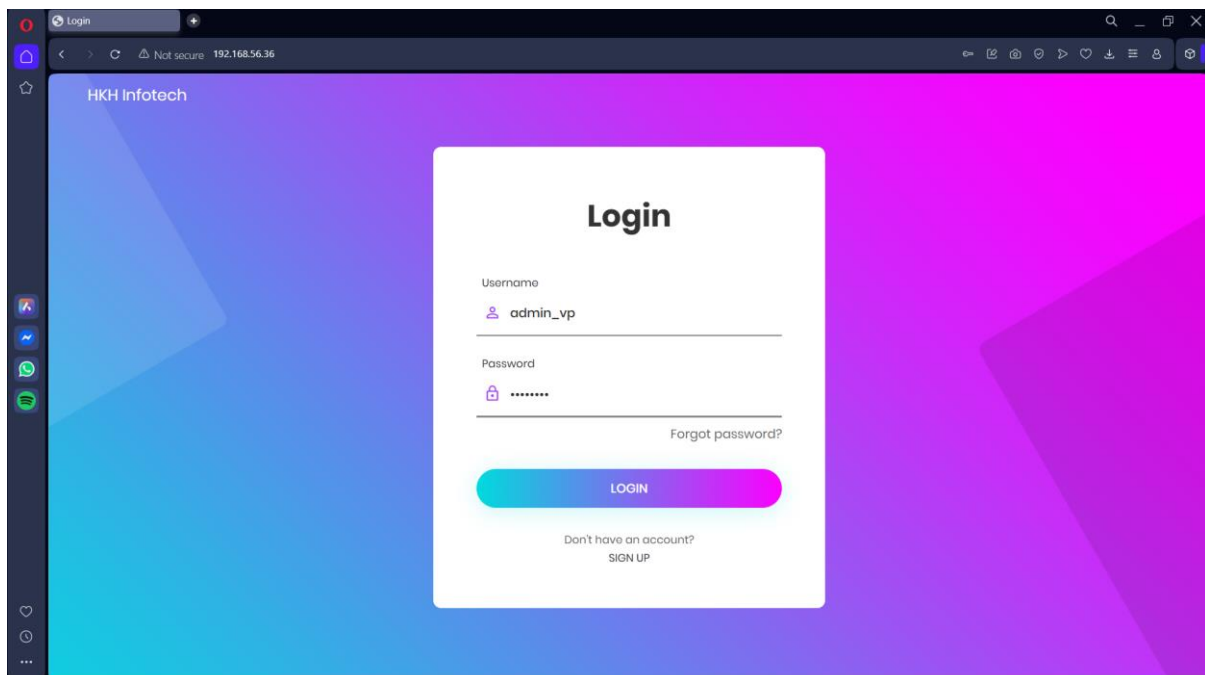
```

root@ubuntu-jammy: /vagrant
root@ubuntu-jammy:/vagrant# ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 02:17:d8:8d:76:f6 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 metric 100 brd 10.0.2.255 scope global dynamic enp0s3
        valid_lft 84273sec preferred_lft 84273sec
    inet6 fd17:625c:f037:2:17:d8:ff:fe8d:76f6/64 scope global dynamic mngtppaddr noprefixroute
        valid_lft 86107sec preferred_lft 14107sec
    inet6 fe80::17:d8:ff:fe8d:76f6/64 scope link
        valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:eb:00:29 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.36/24 brd 192.168.56.255 scope global enp0s8
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:feeb:29/64 scope link
        valid_lft forever preferred_lft forever

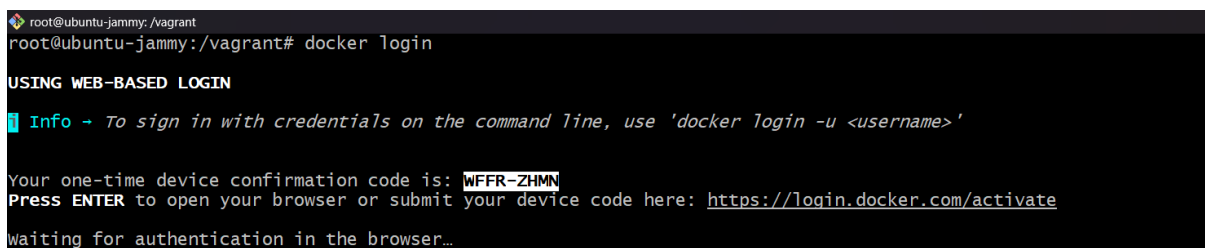
```

- And then access the application, we can get the login.
- User: admin_vp

- Password: admin_vp



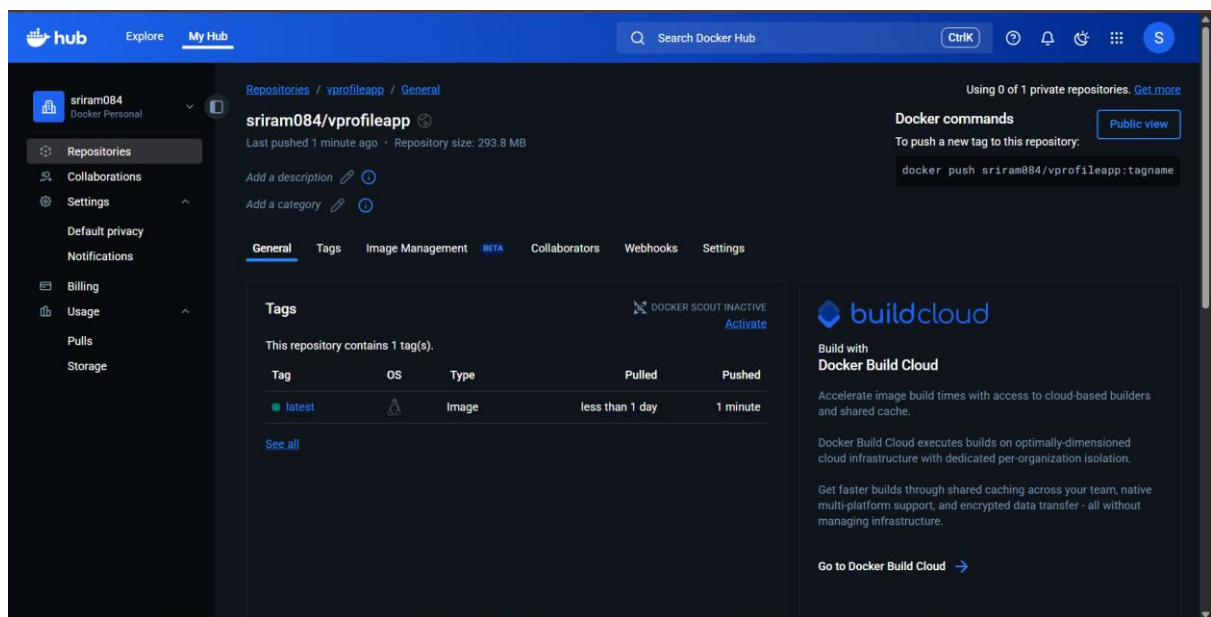
- After that we can push into our docker hub repository by using the docker push.
- Before that we have to login our docker hub account the vm.
- By using the command “docker login”



- After authenticating we can see the available images and we can push them by using the command “docker push <image name>”

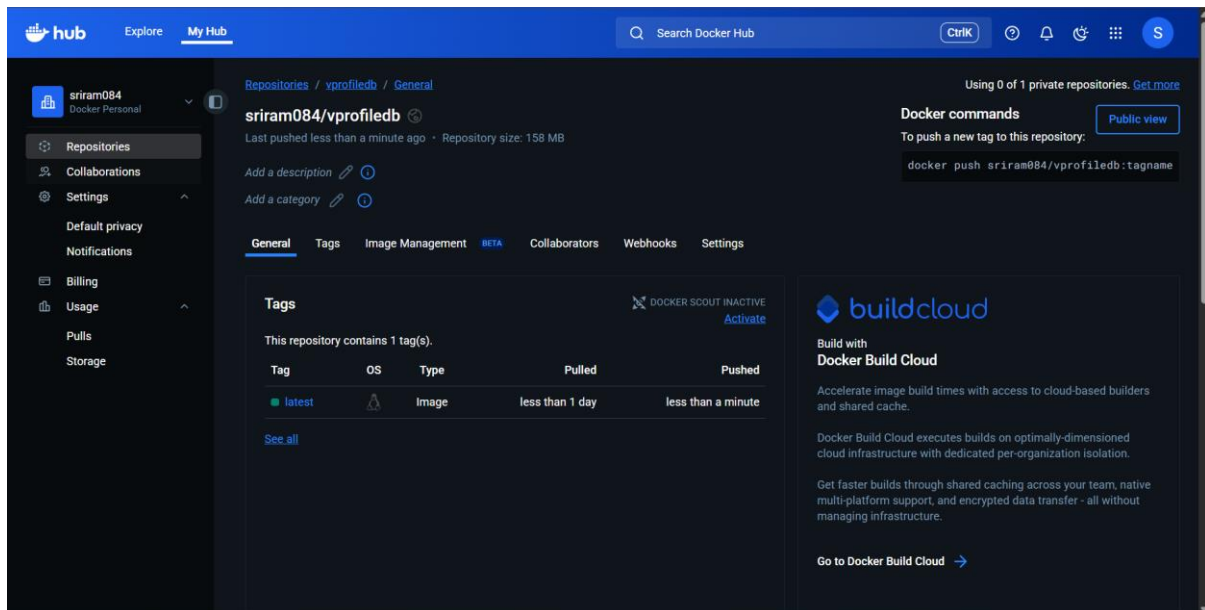
```
root@ubuntu-jammy:/vagrant# docker push sriram084/vprofileapp
Using default tag: latest
The push refers to repository [docker.io/sriram084/vprofileapp]
ddb375431963: Pushed
5f70bf18a086: Mounted from library/tomcat
3a5fe7c2060e: Mounted from library/tomcat
8220f4a58678: Mounted from library/tomcat
57e1c5c4536a: Mounted from library/tomcat
ce4ea9b031b2: Mounted from library/tomcat
00c913b44116: Mounted from library/tomcat
2a8dbe302ea5: Mounted from library/tomcat
f9f52dc133e2: Mounted from library/tomcat
latest: digest: sha256:480f4af1033d9a3accf3f3e093621707a204a78a390768cc7682220384997b3 size: 2619
```

- Here it pushed the “sriram084/vprofileapp” image into the docker hub repo.
- Make sure the image name and the repo name should be same. Then only it can be pushed into repo.



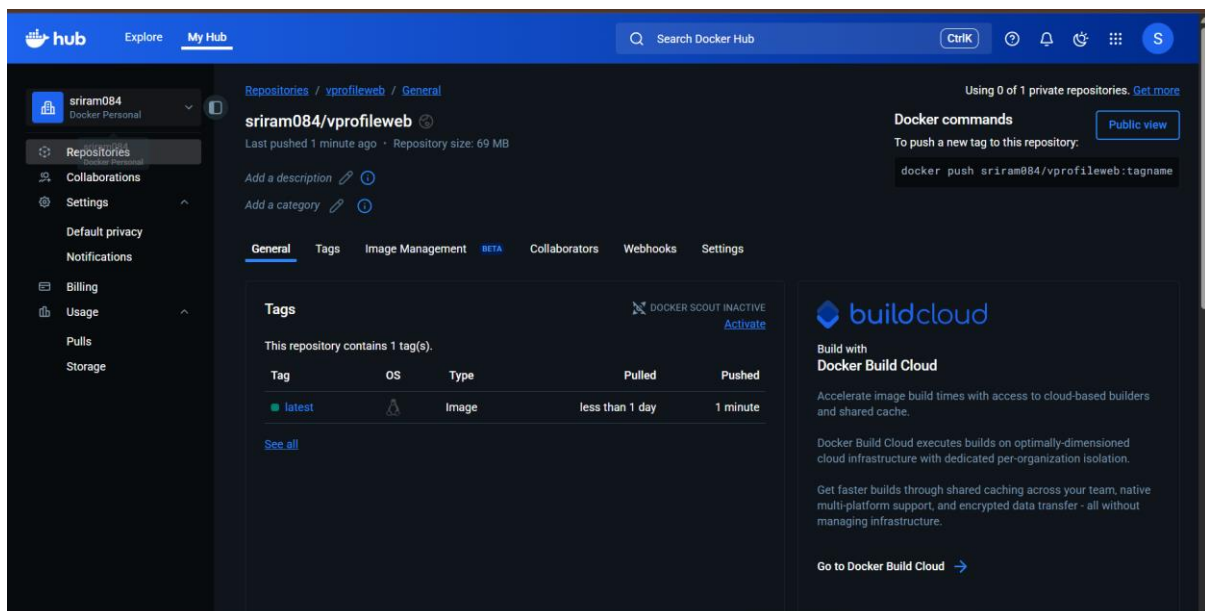
- Here it pushed the “sriram084/vprofiledb” image into the docker hub repo.

```
root@ubuntu-jammy:/vagrant# docker push sriram084/vprofiledb
Using default tag: latest
The push refers to repository [docker.io/sriram084/vprofiledb]
52e15ded51e8: Pushed
30256473ad17: Mounted from library/mysql
b30f75c501b6: Mounted from library/mysql
f5611ea49cae: Mounted from library/mysql
06af60393523: Mounted from library/mysql
6abf55c795bc: Mounted from library/mysql
5353f7b1372e: Mounted from library/mysql
f5cca8023c34: Mounted from library/mysql
c1e3f0059a6c: Mounted from library/mysql
b1a906a58dc2: Mounted from library/mysql
e19b28b0c15e: Mounted from library/mysql
32f7f5f86853: Mounted from library/mysql
latest: digest: sha256:a5b1dd49d1b9bd59eb4089f1d9a6bc3c2dd9198f0e8d5e863ad998a78126aa2d size: 2826
```

- Here it pushed the “sriram084/vprofileweb” image into the docker hub repo.

```
root@ubuntu-jammy:/vagrant# docker push sriram084/vprofileweb
Using default tag: latest
The push refers to repository [docker.io/sriram084/vprofileweb]
9779bbb0fd20: Layer already exists
b52764a3299a: Layer already exists
3ce214e9ebc5: Layer already exists
7d95a4a72e11: Layer already exists
72fa904a482c: Layer already exists
50da593f6222: Layer already exists
8e7d6b511078: Layer already exists
c855abf10cdc: Layer already exists
36f5f951f60a: Layer already exists
latest: digest: sha256:85330b43ede0c25f09bf59da3d3a2308c4b082e24eb1a07cbcc0b153c8fa10a1 size: 2192
```



- Here it pushed the “sriram084/vprofilemq” image into the docker hub repo.

```

root@ubuntu-jammy:/vagrant# docker push sriram084/vprofilemq
Using default tag: latest
The push refers to repository [docker.io/sriram084/vprofilemq]
e9e56ec7db75: Mounted from library/rabbitmq
44475b5d1467: Mounted from library/rabbitmq
ca525453d9f6: Mounted from library/rabbitmq
5b91be1cfe19: Mounted from library/rabbitmq
059390fd8de6: Mounted from library/rabbitmq
1a80d6b5b798: Mounted from library/rabbitmq
a309b31427c6: Mounted from library/rabbitmq
8a11d787ea99: Mounted from library/rabbitmq
f9f52dc133e2: Mounted from sriram084/vprofileapp
latest: digest: sha256:f7d434ea3d17f389c725056267516b9fc37201a0fcd7d7e22cb93404930c996a size: 2200

```

The screenshot shows the Docker Hub interface for the repository `sriram084/vprofilemq`. The repository is public and was last pushed less than a minute ago. The repository size is 106.2 MB. The 'Tags' section shows a single tag named `latest` with an OS of `linux` and a type of `image`. The 'Pushed' time for the `latest` tag is 'less than a minute'. The 'General' tab is selected, and the 'Tags' sub-tab is active. The 'Docker commands' section shows the command `docker push sriram084/vprofilemq:tagname`. The 'buildcloud' section is visible on the right, promoting Docker Build Cloud.

- Here it pushed the “sriram084/vprofilemc” image into the docker hub repo.

The screenshot shows the Docker Hub interface for the repository `sriram084/vprofilemc`. The repository is public and was last pushed less than a minute ago. The repository size is 30.7 MB. The 'Tags' section shows a single tag named `latest` with an OS of `linux` and a type of `image`. The 'Pushed' time for the `latest` tag is 'less than a minute'. The 'General' tab is selected, and the 'Tags' sub-tab is active. The 'Docker commands' section shows the command `docker push sriram084/vprofilemc:tagname`. The 'buildcloud' section is visible on the right, promoting Docker Build Cloud.

